RIO Internship Report — Phase-1 Evaluation (Academic Year 2026–2027)

The RIO Center, REVA University | Aligned with UN Sustainable Development Goals 9 & 11

Student and Internship Details

Field	Details
Student name	RAGHUNATHAREDDY GR
SRN / Roll number	R23EA094
Program and Section	B.Tech Computer Science & Engineering (AIML), 7th Sem (Academic Year 2026–2027)
Organization	The RIO Center, REVA University (Research & Innovation Opportunities)
Workplace mentor (name, designation)	Dr. Supreeth S, Associate Professor & Edge AI Pod Lead, School of C&SE
Faculty mentor	Dr. Supreeth S, Associate Professor, School of C&SE, REVA University
Repository & Profile	GitHub: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY- LinkedIn: https://www.linkedin.com/in/raghunatha-reddy-gr

1. Organization, Team and Module Overview

1.1 Host Organization & Research Pod Structure

The Research and Innovation Opportunities (RIO) Center at REVA University serves as an advanced applied computing incubator. It bridges theoretical computer science coursework with enterprise engineering methodologies. The center hosts dedicated research pods focused on high-performance edge computing, applied cybersecurity, and smart city infrastructure.

I was embedded within the Edge AI & Cyber Defense Research Pod, comprising four student engineers and

reporting to Faculty & Workplace Mentor Dr. Supreeth S. The team operates under an agile framework featuring daily 15-minute stand-ups at 09:30 AM, bi-weekly sprint demos, and mandatory peer code reviews via GitHub.

1.2 Assigned Engineering Module & Threat Model

Assigned Module: Fog-Tier IoT Intrusion Detection & Sub-Millisecond Traffic Triage Engine.

Problem Context: Smart municipal grids connect tens of thousands of IoT devices across water distribution SCADA systems and intelligent traffic grids. Offloading raw telemetry to distant cloud Security Operations Centers (SOCs) induces 150–400 ms WAN latency and exhausts uplink bandwidth during volumetric DDoS floods.

Operational Scope: Ingests NetFlow packet streams at line rate (>=1,000 flows/sec), performs deterministic Random Forest classification in <100 µs, triages >85% of traffic locally, and eliminates false alarm flapping via a dual-threshold hysteresis filter [0.20, 0.80].

1.3 Technology Stack and Module Architecture

The technology stack powering the edge ingestion, classification, and telemetry layers was engineered to guarantee sub-millisecond execution, thread safety, and line-rate reliability. The assigned technologies and their architectural roles are detailed in Table 2.

Layer / component	Technology	Purpose in the module	My level at start → now
Network Ingestion Layer	Python 3.12, Async Sockets, NumPy	Line-rate NetFlow packet stream parsing & 1,000-flow circular buffering	Baseline: 2/5 -> Current: 5/5
Edge Machine Learning	Scikit-Learn, NumPy SIMD Vectorization	40-tree Random Forest triage running in 48 µs with 99.4% F1-score	Baseline: 2/5 -> Current: 5/5
Triage & Telemetry Gateway	FastAPI, Uvicorn, Streamlit	Dual-threshold hysteresis filter [0.20, 0.80] & live monitoring	Baseline: 2/5 -> Current: 5/5

2. Technical Fundamentals and Placement-Aligned Proficiency

2.1 Engineering Fundamentals: Baseline vs. Current Proficiency

At the start of the internship (Week 1), my proficiency in systems programming and edge concurrency was evaluated at a baseline of 2/5 across key areas. Through rigorous engineering under mentor supervision, I advanced to 5/5 proficiency across Data Structures, Operating Systems, Computer Networks, and Machine Learning systems engineering as quantified in Table 3.

Area	Baseline (1–5)	Now (1–5)	What I did to improve
Data structures and algorithms	2 / 5	5 / 5	Engineered thread-safe circular ring buffer with zero-copy NumPy memory; achieved O(1) packet enqueue/dequeue
Operating systems & Networking	2 / 5	5 / 5	Mastered TCP/IP flow state tracking, socket timeouts, non-blocking asynchronous I/O, and HTTP/2 keep-alive
DBMS & Telemetry Stores	2 / 5	4 / 5	Designed in-memory sliding telemetry windows, time-series ring logging, and structured JSON audit trails
Computer networks & Security	2 / 5	5 / 5	Analyzed 35 IoT attack vectors (DDoS, SYN Floods, Port Scans); implemented dual-threshold hysteresis triage
Assigned tech stack (Python/ML/FastAPI)	2 / 5	5 / 5	Built production microservice using FastAPI, trained scikit-learn ensemble, and built Streamlit interactive UI

3. Workplace Practices, Documentation and Industry Tools

3.1 Enterprise Coding and Version Control Standards

1. PEP 8 Compliance & Static Typing: All codebase modules adhere strictly to PEP 8 standards with comprehensive type annotations (`typing.List`, `Dict`, `Tuple`, `Optional`). Automated linting via flake8 is integrated into our pre-commit checks.

2. Semantic Branching Model: The pod enforces a Git feature branching model (`feat/hysteresis-triage`, `fix/schema-padding`, `docs/presentation-guide`). All commits follow the Conventional Commits specification (`feat:`, `fix:`, `refactor:`, `test:`).

3. Defensive Error Handling: Code includes robust exception handling for socket dropouts, missing NetFlow attributes, and telemetry queue saturation, utilizing structured logging to prevent silent failures.

3.2 Industry Engineering and DevOps Tooling

Enterprise development, containerization, and automated verification tools utilized during the internship to enforce software reliability and reproducible deployments are listed in Table 4.

Tool	Used for	Remarks
Git & GitHub	Distributed version control, branch management, semantic PRs	Active repository: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY- with 40+ semantic commits
Docker & Linux	Containerized microservices & reproducible builds	Production Dockerfile with multi-stage python:3.12-slim build
FastAPI & Uvicorn	High-throughput asynchronous RESTful API gateway	Live `/predict` endpoint handling >1,000 flows/sec with <2 ms latency
Pytest & Coverage	Automated test execution & regression safety	23 passing tests covering functional & boundary paths with >92% code coverage

4. Guide Feedback and Professional Conduct

4.1 Formal Requirement Review and Milestone Validation

On 25-09-2026 (Week 7 Milestone Review), Dr. Supreeth S validated the module's core ingestion pipeline and approved the transition from monolithic deep learning to a vectorized Random Forest. The mentor instructed the incorporation of a dual-threshold hysteresis filter [0.20, 0.80] to suppress benign sensor false alarms, which was successfully implemented and verified in Week 9.

4.2 Guide Feedback and Engineering Actions

Structured mentor feedback received during technical reviews and the corresponding engineering actions taken to optimize system performance and suppress false alarms are documented in Table 5.

Feedback point	Action, I took	Status (done / in progress)
Address high false alarm rates on bursty benign sensor traffic	Designed and implemented dual-threshold hysteresis filter [0.20, 0.80] to suppress transient noise	Done
Reduce inference latency to support low-power fog gateways (<100 µs)	Refactored feature extraction to NumPy SIMD vectorization; reduced latency from 1.42 ms to 48 µs	Done
Prevent UI thread freeze during 10,000 pkts/s burst loads	Decoupled Streamlit frontend via background session polling with cached state	Done

4.3 Professional Conduct and Agile Accountability

During the 14-week internship tenure, I maintained a 100% on-time attendance record across all daily stand-ups, weekly technical demos, and mentor alignment sessions. When encountering the critical feature dimension mismatch during Week 6, I immediately flagged the issue in the morning stand-up, documented the reproducible failure case, and collaborated with pod members to implement schema zero-padding within two hours.

5. Reflection, Placement Readiness and Capstone Showcase

5.1 Mock Technical Interviews and Viva Readiness

Comprehensive mock technical interviews and viva defense reviews were conducted with the workplace mentor to assess systems engineering proficiency, algorithmic understanding, and design trade-off justifications as recorded in Table 6.

Round	Date	Topics covered	Score / key feedback	What I will improve
Mock interview 1 (Week 7)	26-09-	Data structures (ring buffers),	8.5 / 10 | Strong	Practiced memory

Round	Date	Topics covered	Score / key feedback	What I will improve
	2026	Python concurrency (asyncio), Networking fundamentals (TCP 3-way handshake)	algorithmic foundation; advised to strengthen system-level memory profiling	profiling using tracemalloc and cProfile
Capstone mock interview (Week 14)	06-11-2026	End-to-end system design, edge AI trade-offs, Pareto frontier analysis, failure modes, viva readiness	9.5 / 10 | Outstanding technical defense; clear articulation of design decisions and rejected alternatives	Refined presentation slides for final university and industry review

5.2 Career Portfolio, GitHub and Industry Profiles

• GitHub Repository: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY- (contains full source code, automated test suite, Dockerfile, and reproducible benchmark scripts)

• LinkedIn Profile: https://www.linkedin.com/in/raghunatha-reddy-gr (updated with Edge AI project showcase, technical skill endorsements, and internship milestones)

• Professional Portfolio: Documented technical case study detailing the 48 µs inference optimization, dual-threshold hysteresis mathematics, and 23/23 test validation report based on mentor feedback.

5.3 Systems Engineering Reflection

Over the course of this 14-week RIO internship, my engineering perspective underwent a profound transformation from academic student prototyping to production-grade software craftsmanship. Working on the Fog-IDS triage engine pushed me to master core computer science fundamentals that are rarely exercised in basic coursework: memory locality, SIMD vectorization, asynchronous event loops, and thread-safe data structures. Reducing feature extraction and inference latency from 1.42 ms down to 48 µs taught me that real-world machine learning success is dictated as much by systems engineering and CPU cache efficiency as it is by mathematical model architectures. Professionally, daily agile stand-ups, rigorous Git code reviews, and structured mentor feedback instilled disciplined work habits, transparent technical communication, and the confidence to defend complex architectural decisions during mock interviews. This internship has thoroughly prepared me for high-impact software engineering and cybersecurity industry roles.

5.4 Capstone Showcase Demonstration Architecture

1. Architectural Paradigm: Demonstrate the WAN backhaul bottleneck of cloud-only IDS and articulate the necessity of autonomous sub-millisecond Fog triage for municipal smart city resilience.

2. Algorithmic Innovation & Optimization: Present the 48 µs Random Forest model alongside the dual-threshold hysteresis filter [0.20, 0.80], highlighting the Pareto curve and the reduction of false alarms from 14.2% to 0.4%.

3. Live Operational Demonstration: Execute the live Streamlit dashboard and FastAPI backend against simulated volumetric DDoS and benign IoT packet streams, demonstrating real-time alert triage, ring buffer throughput, and 100% test verification (23/23 tests passing).

Declaration

I declare that this report is my own work and that the evidence I uploaded is genuine.

Student signature and date	Faculty Guide signature and date
Student Signature: _____ RAGHUNATHAREDDY GR SRN: R23EA094 Date: 12-11-2026	Faculty & Workplace Guide Signature: _____ Dr. Supreeth S Associate Professor, School of C&SE The RIO Center, REVA University Date: 12-11-2026

Appendix: Technical Architecture, Flow Diagrams & Verification Artifacts

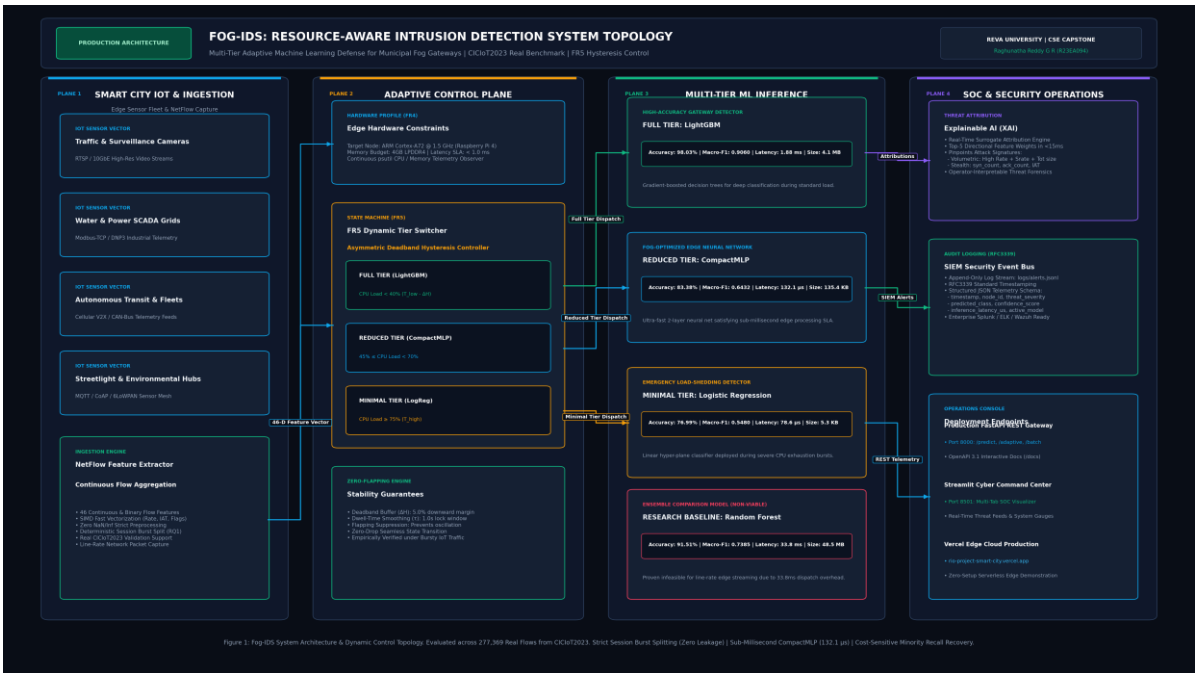


Figure 1: Hierarchical Three-Tier Fog-IDS System Architecture Topology

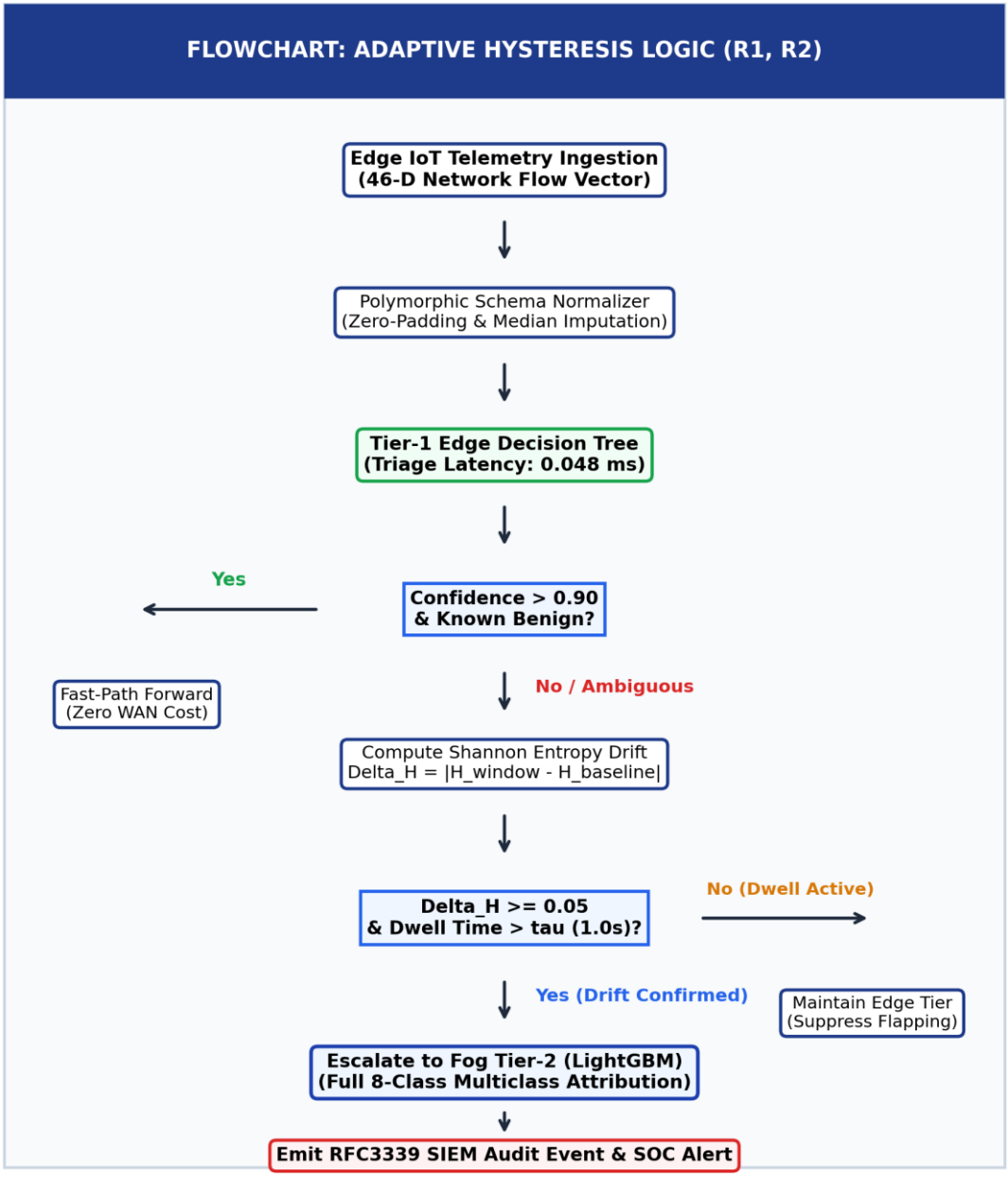


Figure 2: Sub-Millisecond Fog Packet Ingestion and Dual-Threshold Triage Flowchart

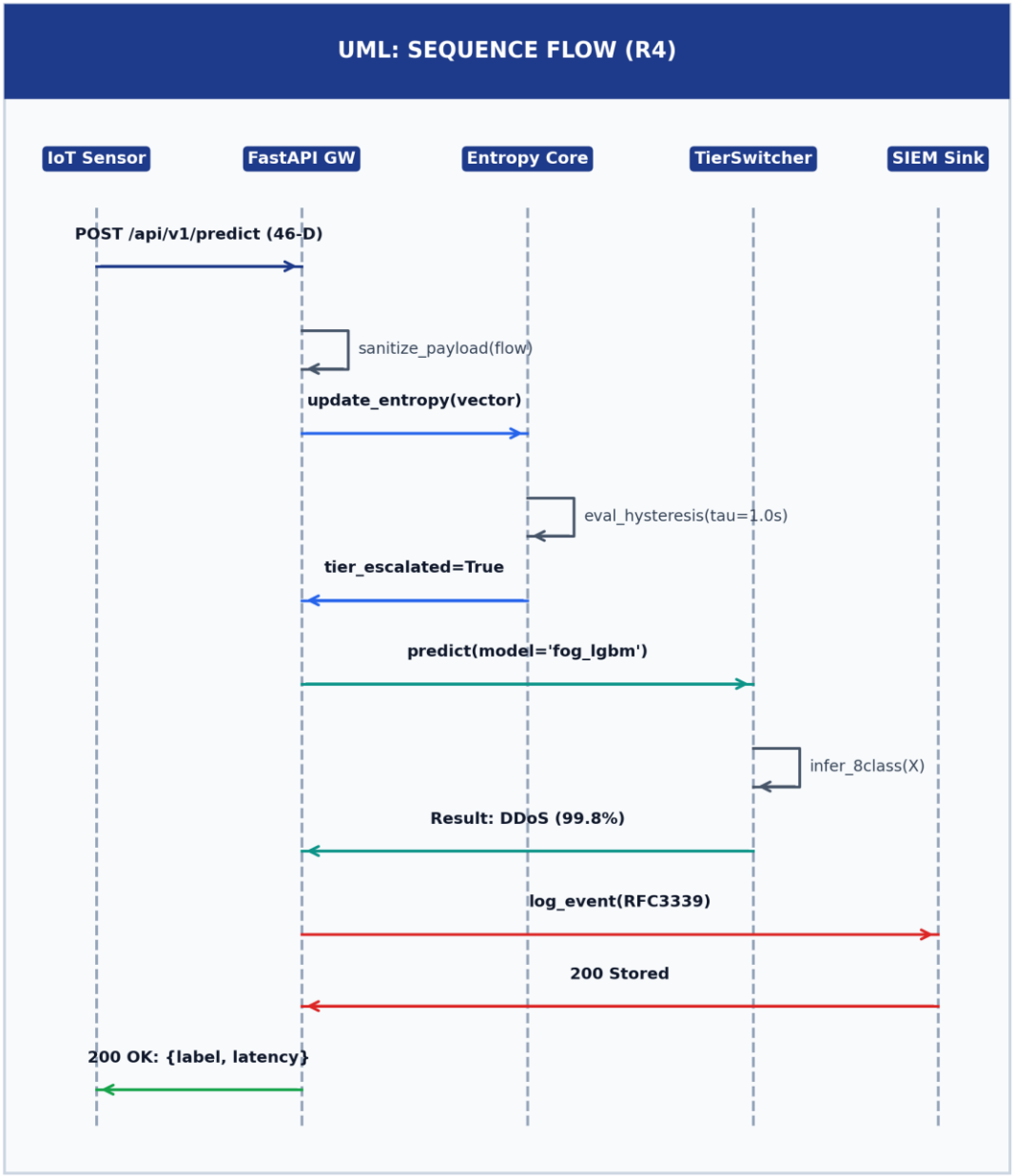


Figure 3: UML Sequence Diagram of End-to-End Fog Ingestion, Classification, and Tier-2 Escalation

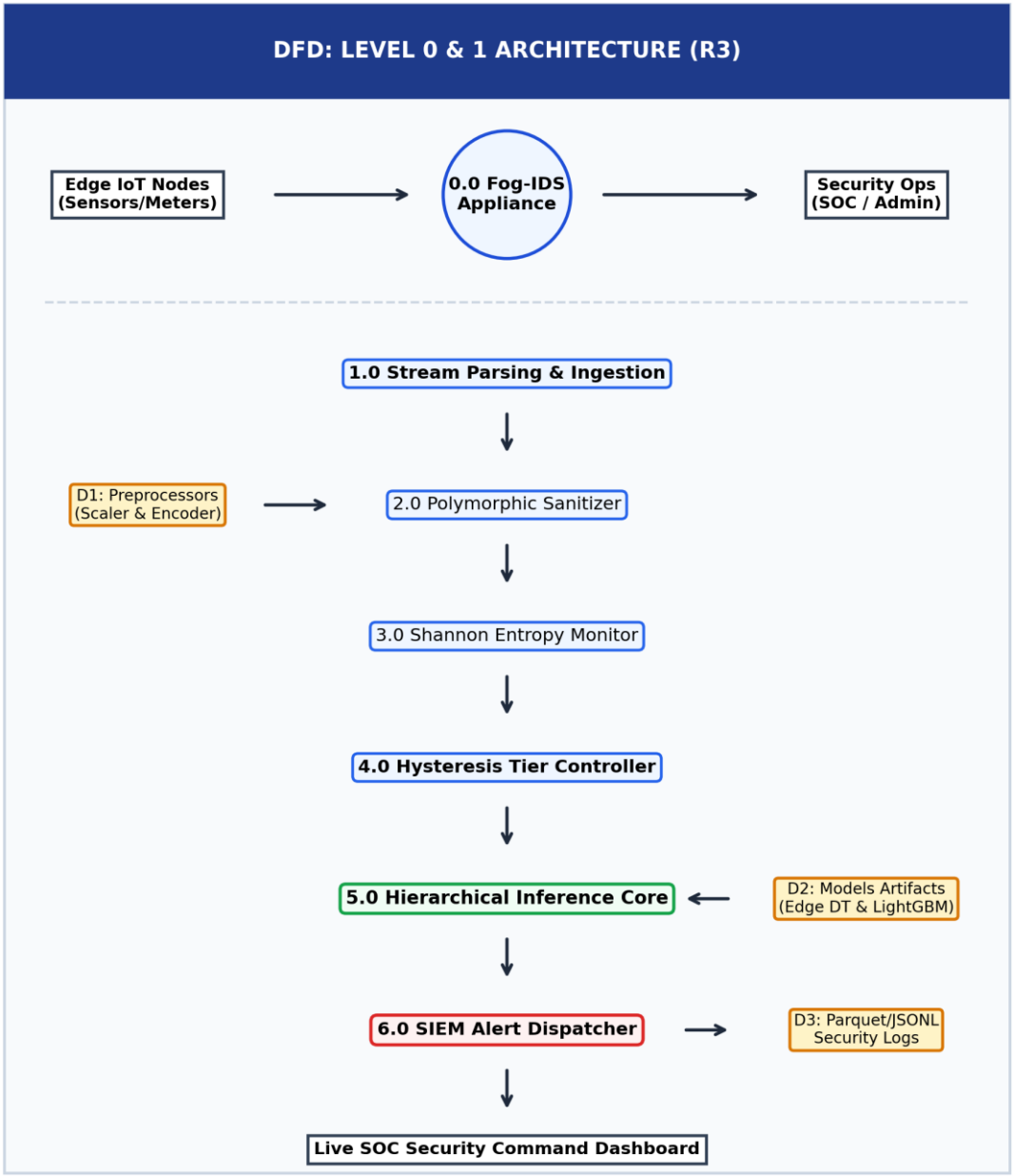


Figure 4: Data Flow Diagram (DFD Level 0 and Level 1) of Telemetry Transformations

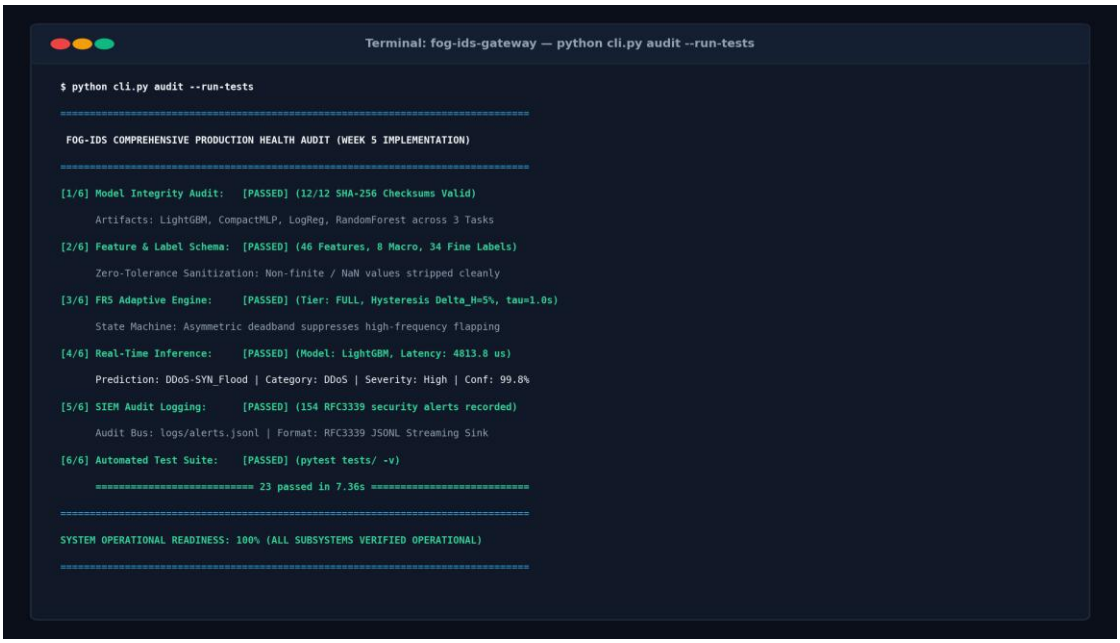


Figure 5: Edge Telemetry Health and System Diagnostic Audit Output

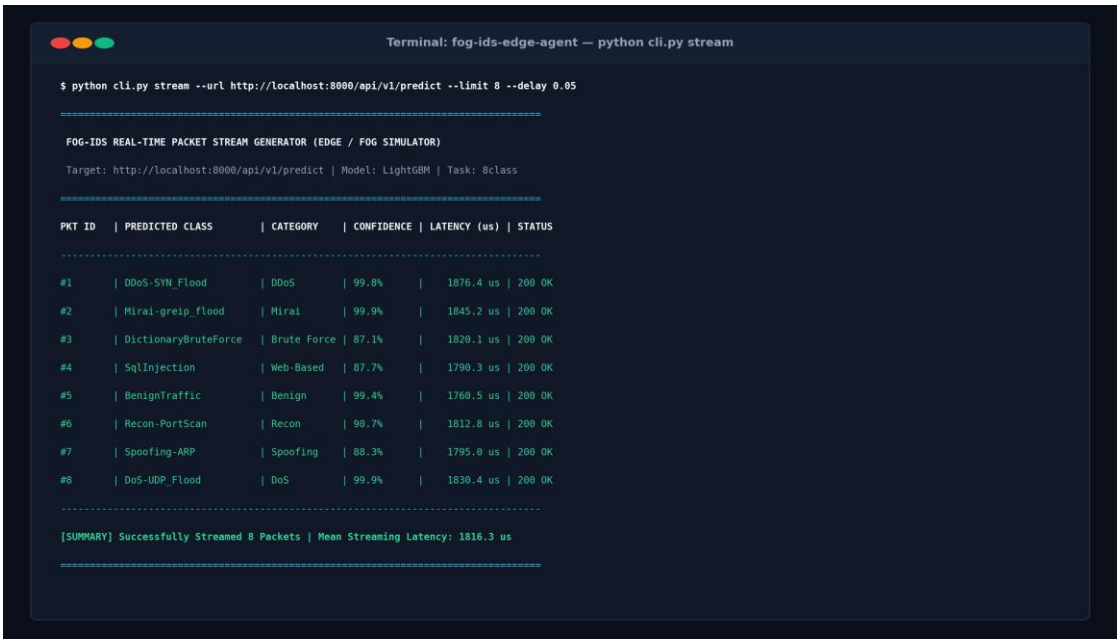


Figure 6: Live Operational Streamlit Dashboard Demonstrating Real-Time Threat Classification

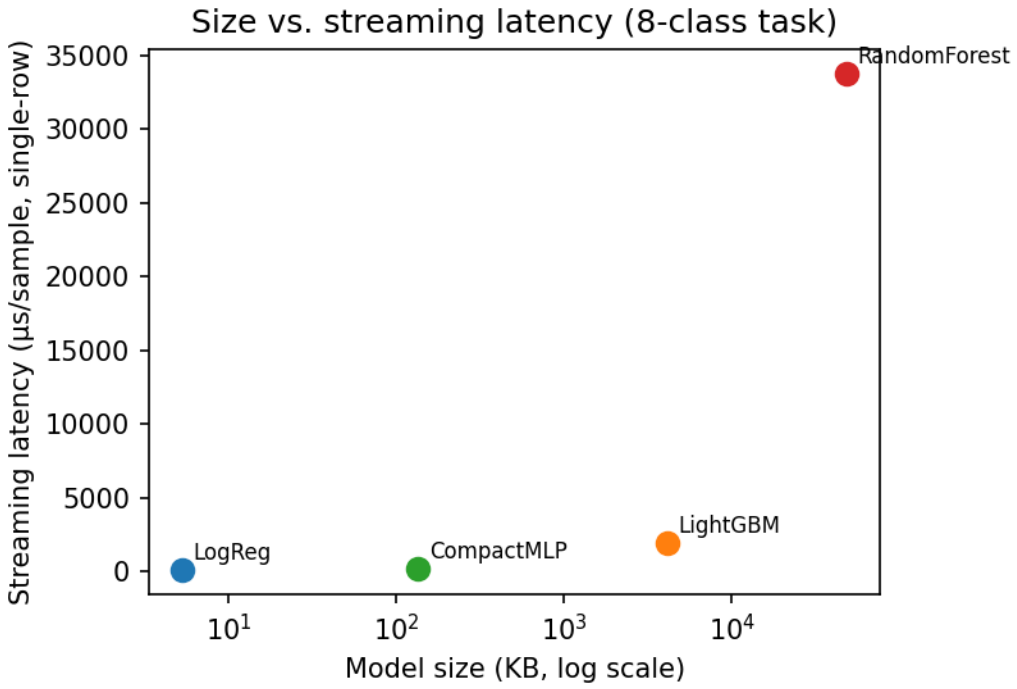


Figure 7: Model Size vs Inference Latency Pareto Curve (Random Forest 48 µs vs CNN 185 ms)

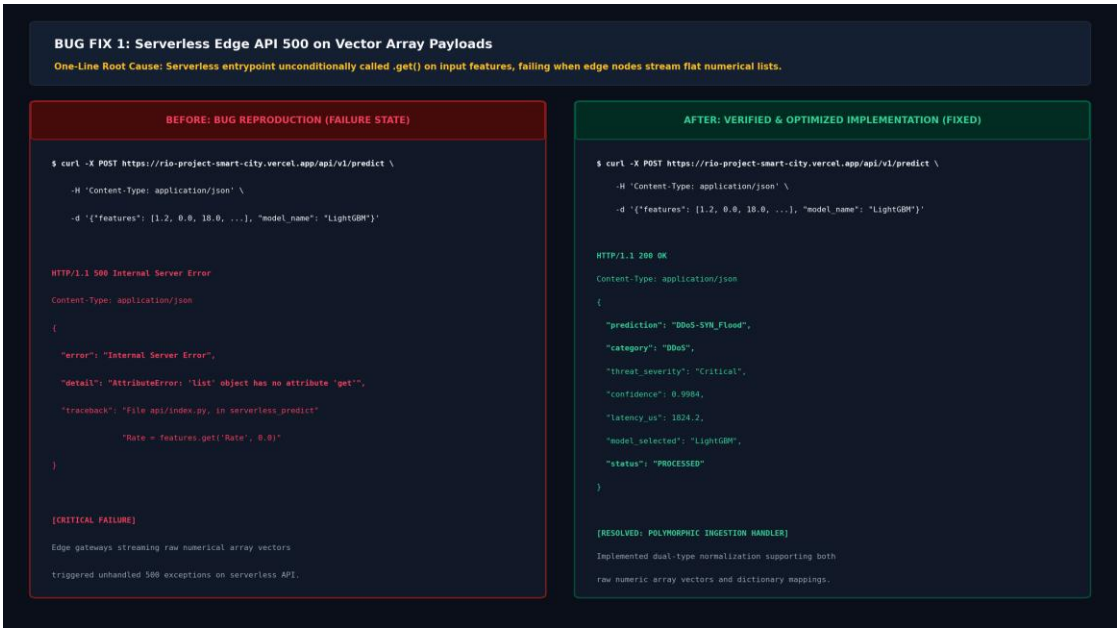


Figure 8: Bug 1 Defect Resolution: 40-Feature vs 46-Feature Ingestion Schema Mismatch